Dynamical System Policy: Robust Visuomotor Policies via Gaussian Dynamic Fields

Anonymous Authors

***Abstract*— Robot manipulation policies can fail when external perturbations push the robot away from its expected motion. Stable dynamical systems offer a natural mechanism for recovery, but fitting expressive feedback fields to continually updated visuomotor predictions can be computationally demanding. We propose *Dynamical System Policy*, which jointly predicts controller commands and end-effector pose trajectories. The commands drive normal execution, while the predicted motion trajectories define a local support region and a Gaussian dynamical field for recovery. When a perturbation moves the robot outside this region, the dynamical field provides feedback that guides the robot back toward supported motion before the policy replans. We evaluate our method on LIBERO-10, RoboMimic Lift, and MimicGen Square, as well as three real-world Franka manipulation tasks and a decentralized Franka–ARX multi robot task. Notably, on LIBERO-10 under external perturbations, our method improves success from 11.84% to 49.64%.**

I. Introduction

Robot manipulation policies [1]–[4] often work well as long as the robot's motion remains close to the motions observed during training. However, disturbances or safety-driven maneuvers can cause the robot to deviate substantially into an unseen state. One straightforward solution is to let the robot observe the new state and replan. However, the policy will usually fail because the disturbed state is out of distribution [5]–[7]. Instead of replanning directly from such a state, we first return the robot to a region supported by the policy's learned motion distribution and then replan. However, fitting stable dynamical systems to visuomotor predictions poses a trade-off: simple, fixed Lyapunov functions can restrict trajectory expressiveness, while more flexible formulations often require additional optimization, making repeated online fitting computationally demanding.

Designing this recovery mechanism requires distinguishing controller commands from the robot's actual motion. During contact, a compliant controller may use a target pose beyond the measured pose to maintain contact force [8]–[10]. For example, when holding down a spring-loaded button, setting the controller target to the current measured pose can reduce the pressing force and allow the button to push the end effector back. Controller commands therefore specify the intended interaction, while measured poses record the resulting motion.

Building on this observation, we introduce *Dynamical System Policy*, which jointly predicts robot commands and their expected motion trajectories. The commands drive nominal execution, while the predicted trajectories describe the expected end-effector motion and provide a geometric reference for detecting deviations and guiding recovery. By aggregating multiple trajectory predictions, we estimate a local support region and construct a Gaussian dynamical field [11]–[14]that guides recovery toward supported motion.

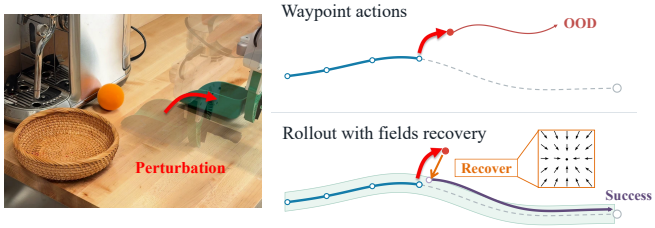


Fig. 1. **Recovery before replanning.** A physical disturbance during manipulation (left). Direct replanning from the disturbed state may fail; our method guides recovery before replanning (right, schematic).

The resulting behavior is simple. As long as the robot remains inside the support region, it executes the policy commands normally through the compliant controller. The geometric model does not alter the commanded motion. If an external perturbation pushes the robot outside this region, the dynamical field provides feedback that guides the robot back toward supported motion. Once the robot has recovered, the policy replans from the new observation.

Multiple predictions also provide useful information at test time. A single predicted trajectory shows only one possible future, whereas several predictions reveal how concentrated or uncertain the policy's expected motion is. We use this information to estimate the support region. Increasing the number of predictions therefore provides additional geometric information without retraining the policy [15]–[17].

We evaluate our method on LIBERO-10 [18], RoboMimic Lift [19], and MimicGen Square [20], as well as three real-world Franka manipulation tasks and a decentralized Franka–ARX multi-robot task. We study both nominal execution and robustness to external perturbations. On LIBERO-10 under perturbations, our method improves success from 11.84% with direct execution to 49.64%, compared with 9.80% for immediate replanning. Increasing the number of paired predictions from one to sixteen further improves nominal success by 8.84%.

Our main contributions are:

- A manipulation policy that jointly predicts robot commands and motion trajectories, preserving native compliant execution while providing geometric information for recovery.
- A test-time mechanism that uses multiple predicted trajectories to estimate a local support region and construct a Gaussian dynamical field around supported motion.
- An online recovery strategy for visuomotor policies that selectively guides the robot back toward supported

motion before replanning, improving robustness across manipulation tasks.

II. RELATED WORK

Generative policies and motion representations. Action-chunking methods generate and execute temporally extended action sequences [1], [2], [21], while waypoint and B-spline representations provide compact descriptions of motion [22], [23]. Other approaches use implicit or discretized action models [3], [24], [25]. Visual objectives provide additional supervision [26], [27]. To select among sampled trajectories, KDPE [15] uses manifold-aware action-density estimation that accounts for position, orientation, and gripper state. Using predicted trajectories for recovery raises a separate issue: commanded targets and measured poses can differ during compliant interaction [8], [9]. Our policy therefore jointly predicts native commands and corresponding future end-effector motion in raw sequences or B-splines. We supervise these predictions with time-aligned commands and measured poses from demonstrations, rather than deriving motion targets from commands. The selected candidate's native commands drive nominal execution, while the predicted geometry supports candidate selection, local support estimation, and recovery control.

Stable dynamical systems and trajectory-induced feedback. Movement primitives [28], SEDS [11], and LPV-DS [12] represent demonstrated motion using attractor dynamics. Lyapunov constraints [29]–[32], invertible transformations [33], and contraction-based architectures [34] construct stable dynamics under specified assumptions, while composed potential fields support sequential behavior [35]. For trajectory-based execution, Spline Policy [36] supports direct spline execution and projection-based distance-field feedback combining attraction toward the curve with motion along it [37]. Under disturbances, the phase assigned by projection may not match task progress. We use the most recent measured pose accepted by the support criterion as the recovery reference, if one is available at recovery onset. Both the reference pose and its recorded phase remain fixed throughout recovery, even as the measured pose changes.

Runtime monitoring and recovery. Expert relabeling and noise-injected demonstrations address compounding errors during learning [5], [6], whereas learned residuals [38] and fast tactile feedback [39] correct actions during execution. For runtime monitoring, Sentinel [40] combines temporal action consistency with vision-language progress assessment, while FAIL-Detect and FIPER use uncertainty-related signals with conformal calibration [41], [42]. For recovery, visual latent-space methods use demonstration distributions to guide corrective actions [7], [43]. Object-centric recovery [44] uses keypoint-manifold guidance and an inverse policy to generate corrective actions. Recovery can also be formulated as return-to-distribution planning [45] or constraint avoidance [46]. To decide whether to intervene, we evaluate the measured state against geometric samples retained from the most recent visual plan. This support estimate measures plan-relative compatibility, not whether recovery will succeed. We additionally check whether the measured motion is consistent with the previously issued command.

III. METHOD

A. Overview

We consider an observation-conditioned manipulation policy executed through a low-level robot controller. At time t, the policy receives observations o_t, while the executor monitors the measured end-effector pose $x_t = (p_t, R_t) \in SE(3)$.

Our policy jointly predicts a controller command sequence and an end-effector pose trajectory. The commands specify the controller inputs, while the predicted poses describe where the end effector is expected to move in response to these commands. As shown in Fig. 2, multiple paired predictions provide a reference trajectory and a local support tube. The executor uses the commands paired with the selected trajectory without geometric correction during normal execution.

When the measured pose leaves the support region and its motion indicates a disturbance, the executor uses the most recent accepted pose and its execution phase as a fixed recovery reference. It then applies recovery feedback and replans from the updated observation (Fig. 3).

B. Paired Geometry–Action Prediction

We construct paired targets from time-aligned controller commands and measured end-effector poses in demonstration windows. The geometric target G_t encodes future measured poses, while U_t contains the corresponding controller commands. At execution, we decode the geometric prediction into a sequence of end-effector poses in the world frame, starting from the current measured pose. Each transition between consecutive poses represents the predicted motion during execution of the corresponding command.

For the end-effector delta-pose interface, each command is $u_j = [\Delta p_j^{\mathrm{cmd}}, \Delta \phi_j^{\mathrm{cmd}}, g_j] \in \mathbb{R}^7$: three normalized translation components, three normalized rotation components, and one gripper command. These specify controller targets, not measured pose changes. During contact, the commanded displacement can therefore differ from the displacement recorded in G_t.

We extend the output of an observation-conditioned diffusion policy to include both geometry and command channels. The network jointly denoises their concatenated, normalized representation using a channel-weighted denoising loss. Both targets are supervised from demonstrations; we do not impose an additional consistency loss between the predicted commands and motion. The resulting policy samples

$$(G_t, U_t) \sim p_\theta(G, U \mid o_t). \tag{1}$$

C. Estimating Local Geometric Support

A single predicted pose trajectory provides a reference path, but does not specify how much deviation should be tolerated during execution. We estimate a local support

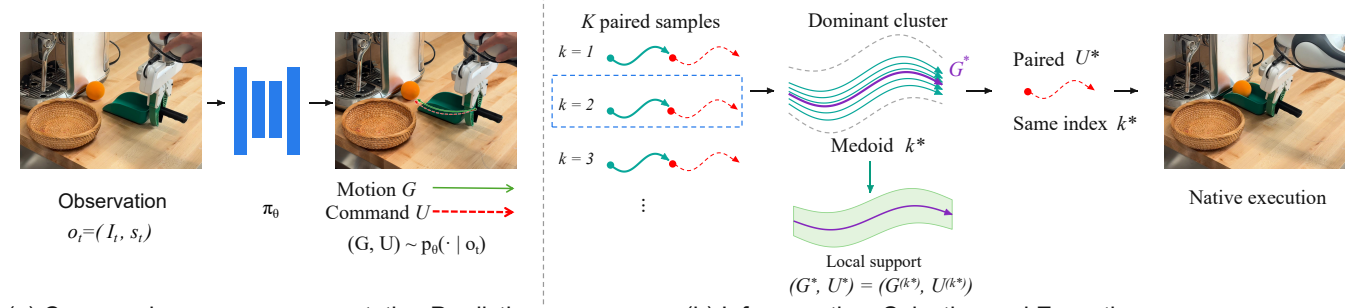


Fig. 2. **Method overview.** (a) The policy jointly predicts an end-effector pose trajectory G and a controller command sequence U. (b) From K paired predictions, we select a representative sampled trajectory from the largest cluster and execute its corresponding commands. The cluster also defines a local support tube.

region from multiple predictions conditioned on the same observation. At each policy query, we draw K samples:

$$(G_t^{(k)}, U_t^{(k)}) \sim p_\theta(\cdot \mid o_t), \qquad k = 1, \ldots, K. \tag{2}$$

Different samples may describe incompatible motions. We use complete-linkage clustering with a distance combining normalized positional and rotational differences between trajectories. Within the largest cluster, we select the sampled trajectory with the smallest average distance to the other members and retain its paired command sequence. Let k^* denote the selected sample:

$$G_t^* = G_t^{(k^*)}, \qquad U_t^* = U_t^{(k^*)}. \tag{3}$$

Figure 2(b) illustrates this selection. We assess support reliability by requiring the selected cluster to contain a minimum fraction of the sampled trajectories.

The execution phase s tracks progress through the command sequence and advances as commands are issued. At each phase, we compare the trajectories in the selected cluster with the reference trajectory. Position deviations perpendicular to the reference path determine the tube's positional radius, while relative rotation errors determine its orientation tolerance. We also use the covariance of the position deviations to construct the recovery field. Let $\mathscr{S}_t(s)$ denote the local support region around the selected reference pose $\bar{x}(s)$ at phase s. The resulting support tube is

$$\mathscr{T}_t = \bigcup_s \mathscr{S}_t(s). \tag{4}$$

During execution, support membership is checked only within the phase neighborhood of the current command segment. A displaced pose near a later segment is therefore not accepted as progress solely because of spatial proximity. The tube describes agreement among the current predictions; it is not a calibrated estimate of the demonstration distribution.

D. Selective Intervention and Recovery Reference

Leaving the estimated support tube does not by itself establish that recovery is needed. Prediction error and normal execution variation can also place the measured pose outside the tube. We therefore trigger recovery only when the measured pose leaves the support region and the measured motion is inconsistent with the previously issued command.

We test whether the measured motion is compatible with the previously issued command. Translation and rotation are evaluated separately:

$$\eta_t^q = \min_{0 \le \lambda \le \lambda_{\max}} \|\Delta q_t - \lambda \Delta \hat{q}_t\|_2, \qquad q \in \{p, \phi\}, \tag{5}$$

where Δp_t is the measured translation increment and $\Delta\phi_t$ is the measured relative rotation vector. We obtain $\Delta\hat{p}_t$ and $\Delta\hat{\phi}_t$ by rescaling the previous command's translation and rotation channels to physical units. This command-based test checks compatibility with the issued action, rather than tracking the predicted trajectory G_t. The bounded scale factor λ accommodates variation in motion magnitude. Innovation is considered excessive when either residual exceeds its corresponding threshold.

When geometric support is reliable, recovery is activated only if the pose leaves the local support region and innovation is excessive. Otherwise, the predicted commands are executed without geometric modification.

During normal execution, we maintain a recovery reference consisting of a measured pose and its execution phase. When the support estimate is reliable, we update this reference whenever the current measured pose lies within the support region for the current phase. States outside this local support do not update the stored pose, even if native execution continues. When recovery is triggered, the most recent accepted pose and its associated phase are used as a fixed recovery reference. The recovery reference remains fixed throughout recovery rather than moving to a spatially closer point at a later phase.

E. Geometry-Based Recovery Dynamics

We construct recovery dynamics from the selected pose trajectory prefix ending at the recovery-reference phase, using its shape to guide motion toward the recovery reference (Fig. 3). In local translational coordinates, the field is represented by a Gaussian-weighted mixture of affine components [11], [12], [47]:

$$f_0(p) = \sum_i h_i(p)\big(A_i(p - c_i) + b_i\big), \tag{6}$$

where each c_i is a sampled prefix position and h_i is a normalized Gaussian weight. Positional support covariances define the position-dependent weights; the full-pose implementation also accounts for orientation differences. Successive

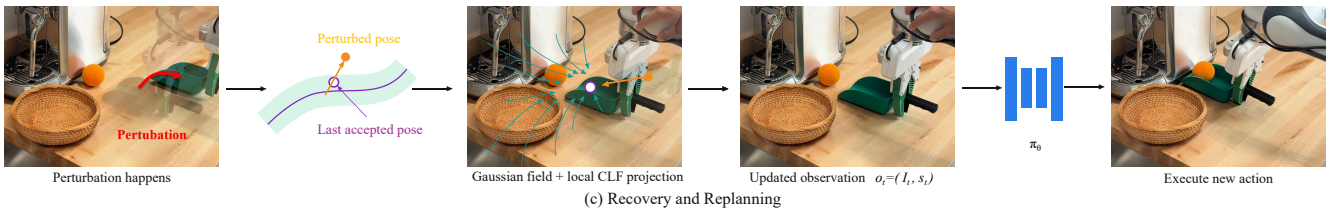


Fig. 3. **Recovery and replanning.** After a disturbance is detected, the robot recovers toward the last accepted pose using a Gaussian recovery field with local CLF projection. Once recovery is complete, the policy replans from the updated observation and resumes execution.

predicted poses define motion targets along the reference path. At additional samples displaced from the path, we add a corrective component pointing back toward the corresponding reference point. When recovery begins, we fit A_i, b_i by regularized least squares, constraining the symmetric part of each A_i to be negative definite. We translate the positional centers so that the prefix endpoint matches the recovery-reference position, and shift the translational CLF coordinates consistently.

Local component constraints alone do not establish convergence of a state-dependent mixture. We therefore apply a local control-Lyapunov projection to the translational recovery field. Let $p_\star$ denote the recovery-reference position and define

$$V(p) = \tfrac{1}{2}(p-p_\star)^\top P(p-p_\star), \qquad P \succ 0, \tag{7}$$

where P is the inverse regularized positional support covariance at the recovery-reference phase, or the identity when unavailable. Unlike the component-specific Gaussian metrics, this CLF metric remains fixed during recovery. Outside the recovery tolerance, the projected field is

$$f(p) = f_0(p) - \frac{[\nabla V(p)^\top f_0(p) + b(p)]_+}{\|\nabla V(p)\|_2^2}\nabla V(p). \tag{8}$$

Here, $[z]_+ = \max(z,0)$ and $b(p) > 0$ specifies the required decrease. We use

$$b(p) = \alpha d(d-r), \qquad d = \sqrt{2V(p)} > r, \tag{9}$$

with $\alpha > 0$ and tolerance radius r. Within the tolerance region, the projection is inactive.

For a fixed recovery reference and metric, the projected translational field satisfies $\nabla V^\top f \le -b$ outside the tolerance wherever the gradient is nonzero. The statement concerns local reference dynamics, not global convergence of the physical robot under contact, actuation limits, or persistent external forces. Orientation feedback uses the relative rotation error to the recovery-reference orientation.

F. Recovery, Reentry, and Replanning

During recovery, the executor pauses the predicted command sequence. It scales the translational field output and rotational feedback into pose increments, normalizes them to the controller's action units, and limits their magnitudes before execution. The gripper does not advance to future predicted commands: a pending transition retains its issued command; otherwise, recovery sends a neutral gripper command. Recovery is complete when position and orientation errors relative to the recovery reference satisfy their tolerances for the required consecutive checks.

Once reentry is confirmed, the executor requests a new paired prediction from the current observation. This refreshes both the command sequence and its geometric support before normal execution resumes, completing the recovery–replanning sequence shown in Fig. 3. Ordinary policy queries also occur when a command chunk finishes; these are distinct from disturbance-triggered replanning requests.

IV. Experiments

We first describe the task set-up and the disturbance design (Sec. IV-A). We then conduct experiments to answer the following questions: (1) Can the policy recover from perturbation-induced deviations and resume task execution? (Sec. IV-B) (2) Is the proposed recovery mechanism effective beyond a single representation? (Sec. IV-C) (3) Does selecting among multiple predicted trajectories improve nominal task success? (Sec. IV-D)

A. Simulation and Real World Task Set-up

Simulation Benchmark. We evaluate our method on LIBERO-10, RoboMimic Lift, and MimicGen Square tasks. For each task, we use 5 random seeds for diffusion policy action sampling and run 50 episodes per policy seed, yielding 250 episodes per task. We use identical action sampling and episode seed schedules across comparisons. Simulation pushes use a configured displacement target of 20 cm while the policy remains active. Within each benchmark, methods share the same disturbance parameters, trigger rules, and time limits. We assess recovery with task success rate, computed by averaging the simulator's binary task-completion indicator over all evaluation attempts. Achieved push distance is measured as the displacement along the push axis at release. The median is reported in Table I.

Single-Arm Tasks. All three single-arm tasks use a Franka robot with RGB images from a third-person camera and a wrist-mounted camera. In ***Pick Cola***, the robot must place the cola container in a designated white dish. In ***Place Bottle and Close Drawer***, the robot must place a medicine bottle inside the drawer and close it. In ***Sweep Ball*** the robot uses a scoop to lift a ball and deposit it into a basket. A trial is considered successful only if the robot completes the entire task. In the real world experiments, pushes are applied manually,

TABLE I

SIMULATION SUCCESS RATES (%). PARENTHESES GIVE MEDIAN ACHIEVED PUSH DISTANCE.

	LIBERO-10		RoboMimic Lift		MimicGen Square	
Method	Nominal	Perturbed	Nominal	Perturbed	Nominal	Perturbed
Raw Action	29.52	9.28 (20.63 cm)	**100.0**	14.8 (21.64 cm)	49.2	11.6 (21.61 cm)
Action BSP	27.96	12.16 (20.61 cm)	**100.0**	14.0 (21.61 cm)	49.2	8.4 (21.31 cm)
Ours (RAW)	**54.16**	**49.64** (20.22 cm)	**100.0**	69.6 (20.27 cm)	**59.6**	**37.2** (19.39 cm)
Ours (BSP)	45.56	46.08 (20.25 cm)	99.6	**71.6** (20.24 cm)	30.0	**37.2** (19.50 cm)

TABLE II

LIBERO-10 ABLATIONS.

Setting	Success (%)
Recovery type ablation (RAW, w/ perturb)	
Direct execution	11.84
Immediate replanning	9.80
Recover, then replan	**49.64**
Test-time scaling ablation (BSP, w/o perturb)	
1 candidate per query	36.72
16 candidates per query	**45.56**

as shown in Fig. 4. Leftward and rightward perturbation directions are matched across conditions. Task success rates are computed from the operator's task-completion records.

Decentralized Two-Robot Cooperation. A Franka and an ARX robot perform complementary subtasks of coffee preparation using separately trained policies. The Franka receives RGB images from two cameras, while the ARX uses a wrist-mounted fisheye camera and the RGB stream from a side-view RealSense camera. Coordination relies on one-way transmission of ARX end-effector information to the Franka for proximity-based avoidance. When the Franka comes too close to the ARX, it moves to a perturbed position and resumes task execution once the ARX clears its path.

B. Recovery Before Replanning

1) Simulation: Table I shows nominal and perturbed performance for both proposed configurations across all three benchmarks. **Raw Action** predicts native controller commands. **Action BSP** predicts B-spline representations of controller commands, which are converted into command sequences for execution. **Ours (RAW)** jointly predicts motion geometry and controller commands, while **Ours (BSP)** uses paired B-spline representations of both. Both proposed configurations use geometric candidate selection and support-aware recovery while preserving native commands during execution. Bold entries indicate the highest observed success rate within each comparison. On LIBERO-10, Ours (RAW) achieves 49.64% under perturbations, compared with 9.28% for Raw Action and 12.16% for Action BSP. Its nominal-to-perturbed decrease is 4.52%, compared with 20.24% for Raw Action. On Lift, all methods achieve 99.6–100.0% nominal success, but perturbed success separates our configurations at 69.6–71.6% from the baselines at 14.0–14.8%.

Achieved push distances differ across methods. On MimicGen Square, Ours (RAW) and Ours (BSP) experience median push distances of 19.39 cm and 19.50 cm, respectively, compared with 21.31–21.61 cm for the baselines. The push-target criterion is not satisfied in 137 attempts for Ours (RAW), compared with zero for Raw Action and four for Action BSP. All such attempts remain in the success-rate denominator. These results therefore evaluate complete closed-loop responses to a shared disturbance protocol, rather than recovery under identical achieved displacements.

The recovery ablation uses the same Ours (RAW) checkpoint with $K = 16$ for all three variants. Direct execution uses this paired model, rather than the Raw Action baseline in Table I. With the candidate count fixed, recovery followed by replanning achieves 49.64% success, compared with 11.84% for direct execution and 9.80% for immediate replanning (Table II), gains of 37.80% and 39.84%. Immediate replanning uses the same intervention trigger, so simply requesting a new prediction after a detected deviation does not reproduce the recovery benefit in this comparison.

This ablation evaluates the recovery procedure as a whole and suggests a benefit from returning to a more suitable state before querying the policy again.

2) Single-Arm Manipulation: The first three rows of Fig. 4 show single-arm task execution following manual perturbations. Both proposed configurations achieve higher success rates under perturbations than the baselines across all three single-arm tasks (Table III). Their perturbed success rates range from 6/10 to 9/10, compared with 2/10 for both baselines on *Pick Cola* and 0/10 on the other two tasks. Under nominal conditions, both proposed configurations achieve 10/10 success on *Pick Cola* and *Sweep Ball*.

3) Decentralized Cooperation: We evaluate decentralized cooperation in a coffee-preparation task, as illustrated in the last row of Fig. 4. We compare Ours (RAW) with Raw Action on the Franka using ten operator-scored trials per method, while keeping the ARX policy fixed. Unlike the single-arm experiments with manually applied pushes, disturbances in this setting arise from robot interactions and avoidance maneuvers during task execution. We therefore evaluate overall task completion without separating trials into nominal and perturbed conditions. Ours (RAW) completes 8 of 10 trials, compared with 4 of 10 for Raw Action (Table IV). This experiment examines whether our method can support continued execution when another independently controlled robot disrupts the task, extending the evaluation beyond externally imposed perturbations.

C. Results Across Action Representations

The improvements under perturbations extend to both RAW and B-spline representations. On LIBERO-10, success increases from 9.28% to 49.64% for RAW and from 12.16% to 46.08% for BSP relative to their respective action-only

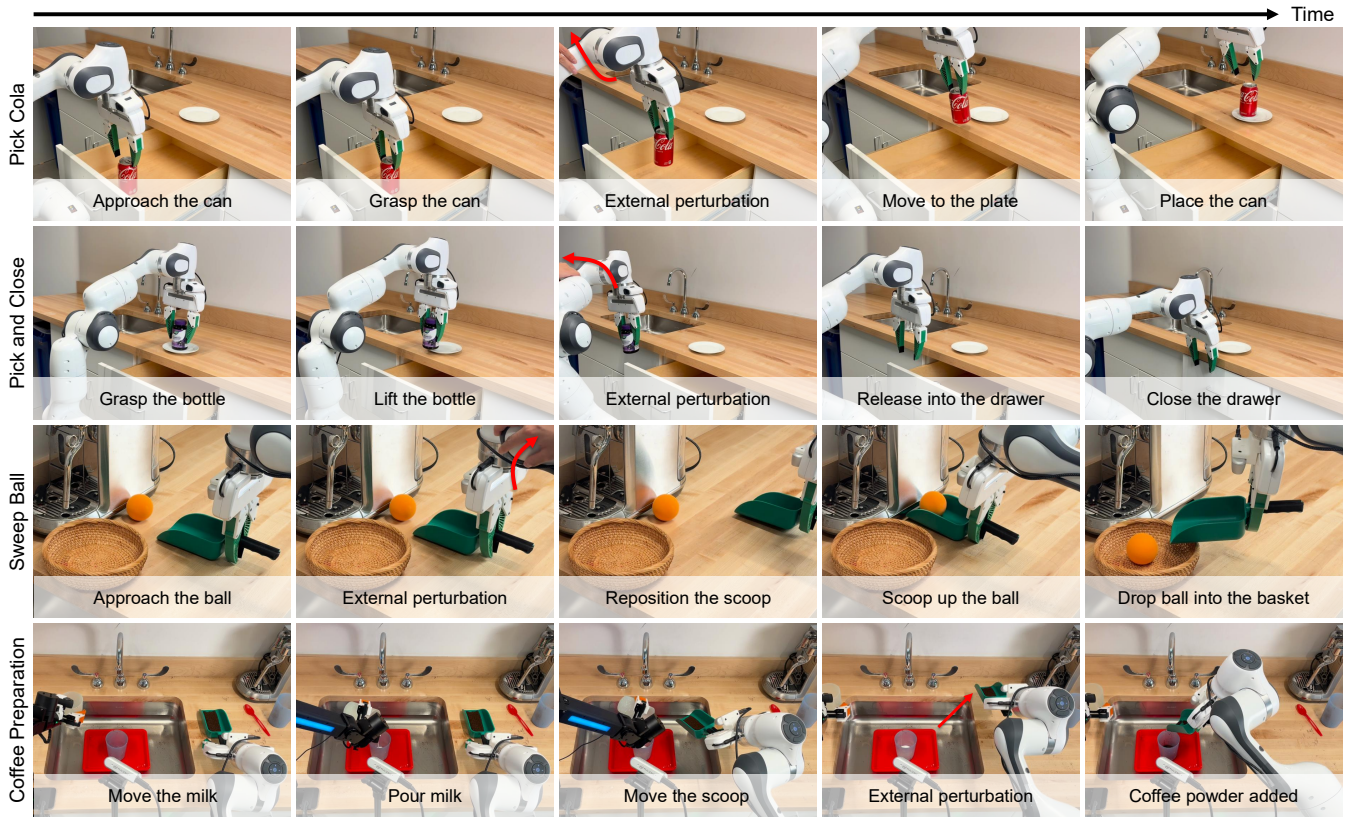


Fig. 4. **Real-world execution under disturbances.** Frames progress from left to right. The first three rows show single-arm task completion following manual perturbations; the bottom row shows decentralized coffee preparation under interaction-induced deviations. Red arrows mark perturbation or avoidance-induced motion.

TABLE III

SINGLE-ARM TASK SUCCESSES OUT OF TEN SCORED TRIALS PER ENTRY.

	Pick Cola		Place Bottle and Close Drawer		Sweep Ball	
Method	Nominal	Perturbed	Nominal	Perturbed	Nominal	Perturbed
Raw Action	8/10	2/10	7/10	0/10	9/10	0/10
Action BSP	9/10	2/10	4/10	0/10	8/10	0/10
Ours (RAW)	**10/10**	6/10	**9/10**	**8/10**	**10/10**	7/10
Ours (BSP)	**10/10**	**8/10**	5/10	7/10	**10/10**	**9/10**

TABLE IV

TWO-ROBOT TASK.

Method	Successes
Raw Action	4/10
Ours (RAW)	**8/10**

baselines. Both representations improve perturbed success on Lift and MimicGen Square (Table I). Across three real-world single-arm tasks, Ours (RAW) succeeds in 21/30 perturbed trials and Ours (BSP) in 24/30, compared with 2/30 for each corresponding baseline (Table III). These results show that the benefit is not limited to a single action representation.

D. Test-Time Candidate Selection

Our method selects the medoid trajectory from the largest cluster and executes its paired command sequence. We evaluate this test-time candidate selection strategy on LIBERO-10 under nominal conditions using the same Ours (BSP) checkpoint for both candidate counts. As shown in Table II, increasing the number of candidates per policy query from $K = 1$ to $K = 16$ improves success from 36.72% to 45.56%, a gain of 8.84%. Both configurations execute the selected commands without recovery, showing that candidate selection can improve nominal task performance independently of the recovery mechanism. The sampled trajectories also support local geometric support estimation.

E. Limitations

Our method has two main limitations. First, the support tube is estimated from only 16 sampled trajectories. The selected medoid represents the retained cluster, but need not lie in a high-density region of the policy distribution. This finite sample set may miss plausible motions or misestimate their spread, so the tube should not be interpreted as a reliable reconstruction of the underlying motion manifold.

Second, satisfying the geometric recovery criterion does not guarantee successful replanning or task completion. A disturbance may change object positions, break a grasp, or alter contact conditions that are not restored by returning the end effector toward its reference pose.

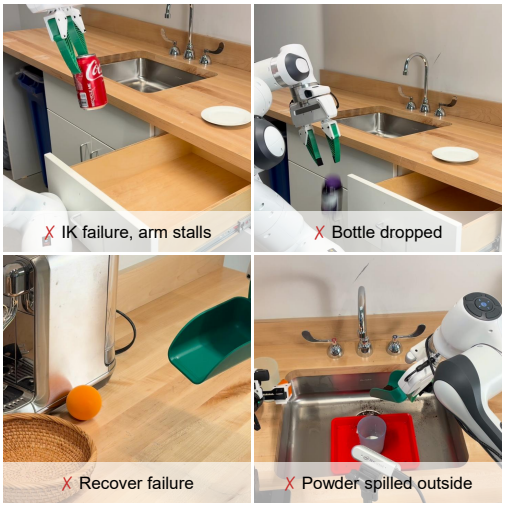


Fig. 5. **Representative failures on the four real-robot tasks.** Panels in reading order: Pick Cola, Place Bottle and Close Drawer, Sweep Ball, and Coffee Preparation.

V. CONCLUSIONS

We presented Dynamical System Policy, a visuomotor policy that jointly predicts controller commands and realized end-effector motion. This paired representation allows nominal execution to preserve the policy's native commands, while the predicted motion provides geometric information for candidate selection, local support estimation, and recovery. When the measured motion departs from this support and becomes inconsistent with the issued command, the robot recovers toward a fixed recovery reference before replanning from the updated observation.

Across simulation and real-world manipulation tasks, this recover-then-replan strategy improves robustness to external and interaction-induced disturbances for both raw and B-spline action representations. In the LIBERO-10 recovery ablation using Ours (RAW), recover-then-replan achieves 49.64% success under perturbations, compared with 11.84% for direct execution and 9.80% for immediate replanning. Increasing the number of test-time candidates also improves nominal performance, showing that the predicted motion geometry is useful beyond recovery alone. These results demonstrate that realized-motion prediction can provide useful test-time structure for robust execution without replacing the policy's native controller commands. Remaining limitations include estimating local support from a finite set of sampled trajectories and the fact that geometric recovery alone cannot restore object state, grasp state, or contact conditions altered by a disturbance.

REFERENCES

[1] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. Burchfiel, and S. Song, "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.

[2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.

[3] N. M. Shafiullah, Z. Cui, A. A. Altanzaya, and L. Pinto, "Behavior transformers: Cloning k modes with one stone," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 22 955–22 968.

[4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "Openvla: An open-source vision-language-action model," in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, 06–09 Nov 2025, pp. 2679–2713. [Online]. Available: https://proceedings.mlr.press/v270/kim25c.html

[5] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.

[6] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "Dart: Noise injection for robust imitation learning," in *Proceedings of the 1st Annual Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, S. Levine, V. Vanhoucke, and K. Goldberg, Eds., vol. 78. PMLR, 13–15 Nov 2017, pp. 143–156. [Online]. Available: https://proceedings.mlr.press/v78/laskey17a.html

[7] A. Reichlin, G. L. Marchetti, H. Yin, A. Ghadirzadeh, and D. Kragic, "Back to the manifold: Recovering from out-of-distribution states," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 8660–8666.

[8] N. Hogan, "Impedance control: An approach to manipulation," in *1984 American control conference*. IEEE, 1984, pp. 304–313.

[9] O. Khatib, "A unified approach for motion and force control of robot manipulators: The operational space formulation," *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, 1987.

[10] Y. Hou, Z. Liu, C. Chi, E. Cousineau, N. Kuppuswamy, S. Feng, B. Burchfiel, and S. Song, "Adaptive compliance policy: Learning approximate compliance for diffusion guided control," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 4829–4836.

[11] S. M. Khansari-Zadeh and A. Billard, "Learning stable nonlinear dynamical systems with Gaussian mixture models," *IEEE Transactions on Robotics*, vol. 27, no. 5, pp. 943–957, 2011.

[12] N. Figueroa and A. Billard, "A physically-consistent bayesian non-parametric mixture model for dynamical system learning," in *Proceedings of The 2nd Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Billard, A. Dragan, J. Peters, and J. Morimoto, Eds., vol. 87. PMLR, 29–31 Oct 2018, pp. 927–946. [Online]. Available: https://proceedings.mlr.press/v87/figueroa18a.html

[13] S. Sun, H. Gao, T. Li, and N. Figueroa, "Directionality-aware mixture model parallel sampling for efficient linear parameter varying dynamical system learning," *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6248–6255, 2024.

[14] S. Sun and N. Figueroa, "Se(3) linear parameter varying dynamical systems for globally asymptotically stable end-effector control," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 5152–5159.

[15] A. Rosasco, F. Ceola, G. Pasquale, and L. Natale, "KDPE: A kernel density estimation strategy for diffusion policy trajectory selection," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Lim, S. Song, and H.-W. Park, Eds., vol. 305. PMLR, 27–30 Sep 2025, pp. 1210–1224. [Online]. Available: https://proceedings.mlr.press/v305/rosasco25a.html

[16] Y. Liu, J. Hamid, A. Xie, Y. Lee, M. Du, and C. Finn, "Bidirectional decoding: Improving action chunking via guided test-time sampling," in *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, Eds., vol. 2025, 2025, pp. 4594–4627. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/file/0d78dd998f7b9ac79604d47a2d79bb0d-Paper-Conference.pdf

[17] N. Ma, S. Tong, H. Jia, H. Hu, Y.-C. Su, M. Zhang, X. Yang, Y. Li, T. Jaakkola, X. Jia, and S. Xie, "Scaling inference time compute for diffusion models," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 2523–2534.

[18] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learn-

ing," *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.

[19] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, "What matters in learning from offline human demonstrations for robot manipulation," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Faust, D. Hsu, and G. Neumann, Eds., vol. 164. PMLR, 08–11 Nov 2022, pp. 1678–1690. [Online]. Available: https://proceedings.mlr.press/v164/mandlekar22a.html

[20] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, "Mimicgen: A data generation system for scalable robot learning using human demonstrations," in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Tan, M. Toussaint, and K. Darvish, Eds., vol. 229. PMLR, 06–09 Nov 2023, pp. 1820–1864. [Online]. Available: https://proceedings.mlr.press/v229/mandlekar23a.html

[21] K. Black, M. Galliker, and S. Levine, "Real-time execution of action chunking flow policies," in *Adv. Neural Inform. Process. Syst.*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, Eds., vol. 38, Main Conference. Curran Associates, Inc., 2025, pp. 33 383–33 407. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/file/300ccb2187dedd4edcc07f7e76d8e553-Paper-Conference.pdf

[22] L. X. Shi, A. Sharma, T. Z. Zhao, and C. Finn, "Waypoint-based imitation learning for robotic manipulation," in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Tan, M. Toussaint, and K. Darvish, Eds., vol. 229. PMLR, 06–09 Nov 2023, pp. 2195–2209. [Online]. Available: https://proceedings.mlr.press/v229/shi23b.html

[23] X. Han, H. Xiong, H. Chen, C. Liu, A. Torralba, Y. Zhu, and Y. Du, "B-spline policy: Accelerating manipulation policies via b-spline action representations," *arXiv preprint arXiv:2607.09648*, 2026.

[24] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit behavioral cloning," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Faust, D. Hsu, and G. Neumann, Eds., vol. 164. PMLR, 08–11 Nov 2022, pp. 158–168. [Online]. Available: https://proceedings.mlr.press/v164/florence22a.html

[25] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto, "Behavior generation with latent actions," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 26 991–27 008. [Online]. Available: https://proceedings.mlr.press/v235/lee24y.html

[26] X. Li, V. Belagali, J. Shang, and M. S. Ryoo, "Crossway diffusion: Improving diffusion-based visuomotor policy via self-supervised learning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 16 841–16 849.

[27] Y. Guo, Y. Hu, J. Zhang, Y.-J. Wang, X. Chen, C. Lu, and J. Chen, "Prediction with action: Visual policy learning via joint denoising process," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 112 386–112 410.

[28] A. J. Ijspeert, J. Nakanishi, H. Hoffmann, P. Pastor, and S. Schaal, "Dynamical movement primitives: learning attractor models for motor behaviors," *Neural computation*, vol. 25, no. 2, pp. 328–373, 2013.

[29] S. M. Khansari-Zadeh and A. Billard, "Learning control Lyapunov function to ensure stability of dynamical system-based robot reaching motions," *Robotics and Autonomous Systems*, vol. 62, no. 6, pp. 752–765, 2014.

[30] N. Figueroa and A. Billard, "Locally active globally stable dynamical systems: Theory, learning, and experiments," *The International Journal of Robotics Research*, vol. 41, no. 3, pp. 312–347, 2022.

[31] J. Z. Kolter and G. Manek, "Learning stable deep dynamics models," *Advances in neural information processing systems*, vol. 32, 2019.

[32] T. Li, S. Sun, S. S. Aditya, and N. Figueroa, "Elastic motion policy: An adaptive dynamical system for robust and efficient one-shot imitation learning," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 9846–9853.

[33] M. A. Rana, A. Li, D. Fox, B. Boots, F. Ramos, and N. Ratliff, "Euclideanizing flows: Diffeomorphic reduction for learning stable dynamical systems," in *Proceedings of the 2nd Conference on Learning for Dynamics and Control*, ser. Proceedings of Machine Learning Research, A. M. Bayen, A. Jadbabaie, G. Pappas, P. A. Parrilo, B. Recht, C. Tomlin, and M. Zeilinger, Eds., vol. 120. PMLR, 10–11 Jun 2020, pp. 630–639. [Online]. Available: https://proceedings.mlr.press/v120/rana20a.html

[34] H. Beik-Mohammadi, S. Hauberg, G. Arvanitidis, N. Figueroa, G. Neumann, and L. Rozo, "Neural contractive dynamical systems," *arXiv preprint arXiv:2401.09352*, 2024.

[35] V. Mengers and O. Brock, "No plan but everything under control: Robustly solving sequential tasks with dynamically composed gradient descent," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 90–96.

[36] M. Tian, Y. Li, S. Liu, A. Ijspeert, and S. Calinon, "Spline policy: A structured representation for robot policies," *arXiv preprint arXiv:2606.07386*, 2026.

[37] Y. Li, N. Darwiche, A. Razmjoo, S. Liu, Y. Du, A. Ijspeert, and S. Calinon, "Geometry-aware policy imitation," in *International Conference on Learning Representations*, C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust, Eds., vol. 2026, 2026, pp. 117 714–117 737. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2026/file/bf235a1d6780afd979f2f81676f43413-Paper-Conference.pdf

[38] X. Yuan, T. Mu, S. Tao, Y. Fang, M. Zhang, and H. Su, "Policy decorator: Model-agnostic online refinement for large policy model," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=e5jGTEiJMT

[39] H. Xue, J. Ren, W. Chen, G. Zhang, Y. Fang, G. Gu, H. Xu, and C. Lu, "Reactive diffusion policy: Slow-fast visual-tactile policy learning for contact-rich manipulation," in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.

[40] C. Agia, R. Sinha, J. Yang, Z. Cao, R. Antonova, M. Pavone, and J. Bohg, "Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress," in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, 06–09 Nov 2025, pp. 689–723. [Online]. Available: https://proceedings.mlr.press/v270/agia25a.html

[41] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. A. Ambrus, H. Nishimura, and M. Itkina, "Can We Detect Failures Without Failure Data? Uncertainty-Aware Runtime Failure Detection for Imitation Learning Policies," in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.

[42] R. Römer, A. Kobras, L. Worbis, and A. Schoellig, "Failure prediction at runtime for generative robot policies," in *Adv. Neural Inform. Process. Syst.*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, Eds., vol. 38, Main Conference. Curran Associates, Inc., 2025, pp. 7631–7670. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/file/0b7cb3b8cc44e652761245537027db44-Paper-Conference.pdf

[43] Z. Sun and S. Song, "Latent policy barrier: Learning robust visuomotor policies by staying in-distribution," in *Adv. Neural Inform. Process. Syst.*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, Eds., vol. 38, Main Conference. Curran Associates, Inc., 2025, pp. 174 280–174 305. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/file/feaf23597f3c93803d1b4230aadc8f44-Paper-Conference.pdf

[44] G. J. Gao, T. Li, and N. Figueroa, "Out-of-distribution recovery with object-centric keypoint inverse policy for visuomotor imitation learning," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 9816–9828.

[45] G. Cideron, B. Tabanpour, S. Curi, S. Girgin, L. Hussenot, G. Dulac-Arnold, M. Geist, O. Pietquin, and R. Dadashi, "Get back here: Robust imitation by return-to-distribution planning," *arXiv preprint arXiv:2305.01400*, 2023.

[46] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg, "Recovery RL: Safe reinforcement learning with learned recovery zones," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4915–4922, 2021.

[47] S. Calinon, F. Guenter, and A. Billard, "On learning, representing, and generalizing a task in a humanoid robot," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 37, no. 2, pp. 286–298, 2007.